

Contentpack Documentation v5

Jeronimo, Uhutown, MrTrobe

16. Januar 2026

Inhaltsverzeichnis

1 Fundamentals	3
1.1 Conceptual Definitions	3
1.2 Technical Definitions	3
1.3 Relations	3
1.4 Functions	4
1.5 Propositional Logic	4
2 Introduction	5
3 Features	6
4 Content Pack Structure	7
5 How to Create a CP?	9
5.1 Registering a Signal System	10
5.1.1 Explanations	11
5.2 Enum Definition / Configuration Options	15
5.3 Creating a Placement Tool	15
5.4 Models and Textures	16
5.4.1 Tint Index	16
5.4.2 Texture Variables	16
5.4.3 Emissive Textures	17
5.5 Creating a Model Definition	18
5.5.1 Explanations	18
5.5.2 Extension File Explanations	20
5.6 Adding Sounds	20
5.6.1 Explanations	20
5.7 Signal Configs for the Interlocking System	21
5.7.1 Folder Structure	21
5.7.2 Reset Setting	21
5.7.3 Default Setting	22
5.7.4 Disable Setting	23
5.7.5 Change Setting	23
5.7.6 Shunting Setting	25
5.7.7 Subsidiary Setting	25
5.8 Signal Bridges	28
5.9 Creating Language Files	30
5.10 Creating Animations	31
5.10.1 Explanations	31
6 Adding the Content Pack to the Game	33
7 Bugs and Errors	33
8 Publishing a Content Pack	34

1 Fundamentals

This section briefly explains basic definitions and the "logic knowledge" required.

1.1 Conceptual Definitions

- **must** is here defined as something that must be done to ensure correct functionality.
- **should** is here defined as something that most users will likely need to do in order to use basic functions.
- **can** is here defined as something that can be done optionally but is not required.

1.2 Technical Definitions

- **STRING** refers to a sequence of characters enclosed by quotes at the beginning and end. For example: `"Text"`
- **INT** refers to an integer without any storage limit. For example: `120`
- **FLOAT** refers to a decimal number without any storage limit. For example: `1.2`
- **BOOLEAN** refers to a truth value (1 bit): either `true` or `false`
- **RELATION** refers to a special STRING that forms a propositional logic formula. These are discussed separately in 1.3
- **PROPERTIES** refers to a special STRING containing certain property names. These are discussed separately in 5.1



If a `...` follows one of these types in the documentation, it indicates a list, meaning multiple different entries can be made, including key-value pairs. For example:

```
"VARIABLE...": "VAR"
```

It can hold any number of pairs:

```
"variable1": "value1",
```

```
"variable2": "value2",
```

```
"variable3": "value3"
```

```
...
```

1.3 Relations

In the following, we introduce important methods and briefly explain the operators used in propositional logic. In general, it is recommended to consult additional online resources for further study.

1.4 Functions

- **config() method** The config method allows signal properties (PROPERTY) to be set as conditions for something, e.g., for other signal properties or signal settings. In other words, something must be present for something else to work. It is used in the signal systems (5.1). Multiple signal properties can also be logically combined. See propositional logic 1.5.

"config(PROPERTY.ENUM) "

- **with() method** The with method allows signal properties (PROPERTY) and their associated settings (ENUM) to be set as conditions for submodels. In other words, something must be present for something else to work. It is used in the model definition (5.5). Multiple signal properties can also be logically combined. See propositional logic 1.5.

"with(PROPERTY.ENUM) "

- **hasandis() method**¹ The hasandis() method allows truth-value properties (BOOLEAN-PROPERTY) to be set as conditions for submodels. The method checks whether the respective BOOLEAN-PROPERTY is present or not and, unlike the has() method, refers only to the current model part. In other words, something must be present for something else to work. It is used in the model definition (5.5). Multiple signal properties can also be logically combined. See propositional logic 1.5.

"hasandis(BOOLEAN-PROPERTY) "

- **has() method** The has() method allows truth-value properties (BOOLEAN-PROPERTY) to be set as conditions for submodels. The method checks whether the respective BOOLEAN-PROPERTY is present or not and, unlike the hasandis() method, refers only to another model part. In other words, something must be present for something else to work. It is used in the model definition (5.5). Multiple signal properties can also be logically combined. See propositional logic 1.5.

"has(BOOLEAN-PROPERTY) "

1.5 Propositional Logic

- **&&** (AND operator)
- **||** (OR operator)
- **()** (Parentheses)
- **!** (Negation)

A	B	&&	A	B		A	B	C	(A B) && C	A	B	C	(A && B) C
w	w	w	w	w	w	w	w	w	w	w	w	w	w
w	f	f	w	f	w	w	w	f	f	w	w	f	w
f	w	f	f	w	w	w	f	w	w	w	f	w	w
f	f	f	f	f	f	w	f	f	f	w	f	f	f
						f	w	w	w	f	w	w	w
						f	w	f	f	f	w	f	f
						f	f	w	f	f	f	w	w
						f	f	f	f	f	f	f	f

A	!A
w	f
f	w

Tabelle 1: Logical combination of truth values.

¹A Boolean property is a signal property whose **enumClass** is BOOLEAN. More details in 5.1.

2 Introduction

In this documentation, we will explain how a Content Pack, or CP for short, is structured. We will start by briefly presenting the features you can use for your own signals. Then, we will explain the structure of the CP and introduce its individual elements. Finally, we will cover how to integrate the CP into the game, how to handle bugs and errors, and how to publish the finished CP.

For creating CPs, we recommend using the following programs as tools, though you are, of course, free to use others.

- **BlockBench** – Modeling and texturing
- **GIMP** – Image editing, creating textures
- **Visual Studio Code** – Editing JSON files
- **7-zip** – Creating and managing ZIP files
- **Audacity** – Editing sound files and saving them in .ogg format

Our mod is based on the idea that signals are not modeled as a whole, but broken down into smaller parts that are either always present or can be optionally selected. These parts are then assembled into a full signal in-game by the system. This approach allows signals to be configured freely according to user preferences. Another fundamental structure of our mod is that textures can be swapped to display different signal indicators (e.g., lamps, letters on Zs2 or digits on Zs3, or the texture of the mast signs).

Another key component of our mod is the interlocking system (Stellwerk), which allows you to control signals and other elements via routes. For CP signals to work with the interlocking, you need to specify which signal indicators should be displayed in which situations.

Due to this level of customization, it is not always straightforward for newcomers to get started with the system, even though we have made an effort to keep it relatively simple. If you have any questions, you can contact us at any time. Theoretically, there are also plans to create an editor that helps generate the JSON files, but this is still far in the future, as our current capacity is limited and the current priority is the signal mod itself.

3 Features

For simplicity, this guide will not cover the general features of the Open-Signals mod, but will focus only on the features relevant to CPs. The following options are available to you:

- Add new models
- Add new textures
- Use tint indices
- Add and configure changeable textures, e.g., for lamps
- Add new placement tools
- Add a damage value to the item and configure it individually
- Configure new signals as you like
- Render and configure custom text on signals, e.g., size, color, positioning
- Add sounds to the signals
- Define a Redstone output
- Make signals compatible with the interlocking system
- Express your creativity with signals
- Add custom signal bridges (blocks)
- Animate models/signals

4 Content Pack Structure

A CP is generally structured like a standard Minecraft resource pack. However, we include additional folders for our systems. A detailed explanation of what each folder is responsible for will be provided in the next chapter. You can download a CP template [here](#).

```
assets/
├── opensignals/
│   ├── animations/
│   ├── blockstates/
│   ├── enumdefinition/
│   ├── lang/
│   ├── modeldefinitions/
│   ├── models/
│   │   ├── block/
│   │   │   └── contentpackname/
│   │   ├── item/
│   │   │   └── contentpackname/
│   ├── recipes/
│   ├── signalbridge/
│   ├── signalconfigs/
│   │   ├── change/
│   │   ├── default/
│   │   ├── reset/
│   │   ├── shunting/
│   │   ├── subsidiary/
│   │   └── subsidiaryenums/
│   ├── signalsystems/
│   ├── sounds/
│   │   └── contentpackname/
│   ├── textures/
│   │   ├── blocks/
│   │   │   └── contentpackname/
│   │   ├── items/
│   │   │   └── contentpackname/
│   └── tools/
```

Explanations

- **assets** is the base folder. Must exist.
- **opensignals** identifies that the CP belongs to the OpenSignals mod. Must exist.
- **animations** contains animation data if a signal is animated.
- **blockstates** contains vanilla Minecraft blockstate files for the signal bridge blocks.
- **enumdefinition** contains one or more files that add new configuration options. Should exist.
- **lang** contains translation files for your settings and signals. Should exist.
- **modeldefinitions** contains the files that control the behavior of your models. Should exist.
- **models** contains all models for the items and signal parts that make up the final overall model. Should exist.
- **recipes** contains crafting recipe files for the signals. Follows vanilla Minecraft logic.

- **signalbridge** contains one or more files to register signal bridge blocks.
- **signalconfigs** contains possible configuration files for the interlocking systems. Can exist.
- **change** contains configurations used when setting a route. Can exist.
- **default** contains configurations used when setting an end block. Can exist.
- **reset** contains configurations used when clearing routes. Can exist.
- **shunting** contains configurations used when setting shunting routes. Can exist.
- **subsidiary** contains configurations for additional signals or signals with special functions. Can exist.
- **subsidiaryenums** is where new additional signals or signals with special functions can be defined in one or more files. Can exist.
- **signalsystems** contains all definitions for the signal systems. Should exist.
- **sounds** contains all sounds and their configurations. Can exist.
- **textures** contains all textures required by your block and item models. Should exist.
- **tools** contains all definitions for custom selection items. Can exist.



To increase compatibility with other CPs and our own models and textures, a subfolder with your content pack name should be created in the folders **models/block/**, **models/item/**, **textures/blocks/**, and **textures/items/**, as shown in the structure above. Note that you must also specify this in your models!



Note: Except for the folders mentioned above, no other folder should contain a subfolder with your CP name, as files in these subfolders will not be used by our system.

5 How to Create a CP?

In this chapter, we explain how to create a CP. This includes:

- Registering signal systems (5.1)
- Configuration options (5.2)
- The selection item (5.3)
- Models and textures (5.4)
- Model definitions (5.5)
- How to add sounds to the signals (5.6)
- How to make signals compatible with the interlocking system (5.7)
- How language files work (5.9)
- How to animate signals (5.10)



In our opinion, the most sensible approach is to first think about what signals you want to add. Then, create the model and the associated textures, including any textures that need to be swapped, e.g., for lamps. If you are using sounds, it is recommended to add them as early as possible so that all necessary „assets“ are available from the start. For the signals themselves, your selection item must be registered, and the signal system must be registered with all configuration options. Afterwards, the individual model parts must be assembled and their behavior defined according to the settings. Optionally, the signal system can then be made compatible with the interlocking system, and finally, language files need to be written so that the display names in Minecraft are correct.



Important preliminary information:

To avoid conflicts with other CPs, it is advisable to give your signal systems unique names, e.g., „jeronimo_hvsignal“. Therefore, it is also recommended to create subfolders for your models and textures using your individual content pack name. In many other areas, it is important to ensure that names and values are unique to prevent conflicts with the main mod or other CPs. Please note that these names are internal names only and do not necessarily correspond to the display names, which are created via language keys and language files.

5.1 Registering a Signal System

If you want to create a new signal system, you must register it. To do this, create a JSON file in the **signalsystems/** folder with the name of your signal system. This file contains important information about the system's settings as well as the properties (PROPERTY) of the signals. The file name serves as the internal name of the signal system and is used in various places, including the interlocking system settings (5.7). Additionally, the model definition file (5.5) must have the same name.

```
1 {
2   "systemProperties": {
3     "placementToolName": "NAME",
4     "canLink": "BOOLEAN",
5     "isBridgeSignal": "BOOLEAN"
6     "defaultItemDamage": "INT",
7     "defaultHeight": "INT",
8     "customNameRenderHeight": "FLOAT",
9     "offsetX": "FLOAT",
10    "offsetY": "FLOAT",
11    "signScale": "FLOAT",
12    "signWidth": "FLOAT",
13    "autoscale": "BOOLEAN",
14    "textColor": "INT",
15    "doubleSidedText": {
16      "RELATION...": "BOOLEAN"
17    },
18    "colors": [
19      "COLOR..."
20    ],
21    "renderHeights": {
22      "RELATION...": "FLOAT"
23    },
24    "signalHeights": {
25      "RELATION...": "FLOAT"
26    },
27    "sounds": {
28      "RELATION...": {
29        "name": "SOUND",
30        "length": "INT"
31      }
32    },
33    "redstoneOutputs": {
34      "RELATION..." : "PROPERTY"
35    },
36    "remoteRedstoneOutputs": {
37      "RELATION..." : "PROPERTY"
38    },
39    "customRenderBoundingBox": ["x1", "y1", "z1", "x2", "y2", "z2"]
40  },
```

```

1  "seProperties": [
2    {
3      "name": "PROPERTY",
4      "enumClass": "ENUM-LIST",
5      "defaultState": "STATE",
6      "changeableStage": "STAGE",
7      "autoname": "BOOLEAN",
8      "itemDamage": "INT",
9      "dependencies": "RELATION..."
10   },
11   {
12     "name": "PROPERTY",
13     "defaultState": "BOOLEAN",
14     "changeableStage": "STAGE",
15     "autoname": "BOOLEAN",
16     "itemDamage": "INT",
17     "dependencies": "RELATION..."
18   }
19 ]
20 }

```

Listing 1: Structure of the Signal System.

5.1.1 Explanations

- **systemProperties** specifies the basic signal system settings. Must be present.
- **placementToolName** specifies in which selection item the signal system is registered. Must be present and given as a string. The internal name of the item must be used here. Example:

`"placementToolName": "signalplacer"`



If you create your own placement tools, please make sure that the names are lowercase, contain no spaces, and no underscores internally.

- **canLink** indicates whether the signal can be linked to a controller or interlocking system. Optional. Default is true, but it can also be set to false.

`"canLink": false`

- **isBridgeSignal** indicates whether the signal is intended for signal bridges. Optional. Default is false.

`"isBridgeSignal": true`

- **defaultItemDamage** specifies how much damage the item takes by default when placing signals. This is in addition to the damage from the selected signal properties. An item has 100 damage points. Optional. Default is 1. Only integers ≥ 0 are allowed.

`"defaultItemDamage": 1`

- **defaultHeight** specifies the height of the signal. Should be present. Default is 1. Only integers ≥ 0 are allowed, where 0 corresponds to a height of 1 block.

`"defaultHeight": 1`

- **customNameRenderHeight** vertical offset of the rendered text. Optional. Activates rendered text if present. Decimal values are allowed.

`"customNameRenderHeights": 1.4`

- **offsetX** horizontal offset of the rendered text. Optional. Linked to **customNameRenderHeight**. Default is 0. Decimal values allowed. Approximately in pixels. Positive values move text right, negative left from the center.

"offsetX": -3.5

- **offsetY** forward/backward offset of the rendered text. Optional. Linked to **customNameRenderHeight**. Default is 0. Decimal values allowed. Approximately in pixels. Positive moves backward, negative forward from the center.

"offsetY": 1.5

- **signScale** specifies the scaling factor of the text size. Optional. Linked to **customNameRenderHeight**. Default is 1. Decimal values allowed.

"signScale": 3.5

- **signWidth** specifies the width of the text field in font pixels. Optional. Linked to **customNameRenderHeight**. Default is 22. Decimal values allowed.

"signWidth": 22

- **autoScale** specifies whether the rendered text should be scaled automatically. Optional. Default is false, but can be set to true.

"autoScale": false

- **textColor** is the color of the rendered text. Optional. Only decimal values (16777215 = white) are allowed. The value can be found, e.g., on Hexadecimal / Decimal Colors.

"textColor": 16777215

- **doubleSidedText** specifies whether the text should be displayed on the opposite side of the signal. Optional. Multiple entries can be made.

"doubleSidedText": {"config(SH_HIGH.HIGH)": true}

- **colors** specifies colors for tint indices in the models. Optional. The first number corresponds to index 0, the second to index 1, etc. Only decimal color values (16777215 = white) are allowed. See 5.4 for more about tint indices.

"colors": [16777215, 0]

- **renderHeights** specifies under which conditions (RELATION) the standard height (FLOAT) of the rendered text should be overridden. Optional. Multiple entries allowed. Decimal values allowed.

"renderHeights": {"config(SH_HIGH.HIGH)": 2.373}

- **signalHeights** specifies under which conditions (RELATION) the defaultHeight of the signals should be overridden. Optional. Multiple entries allowed. Only integers ≥ 0 allowed.

"signalHeights": {"config(SH_HIGH.HIGH)": 2}

- **sounds** specifies under which conditions (RELATION) a sound (SOUND) should be played and repeated (INT). Optional. Multiple entries allowed. SOUND is the internal sound name. INT gives the duration in ticks (20 ticks = 1 second). Only required if the sound should repeat. Should match the exact length of the sound.

```

1 {
2   "sounds": {
3     "config(ac_blink_light.TRUE)": {
4       "name": "andreas_cross",
5       "length": 84
6     }
7   }
8 }

```

- **redstoneOutputs** specifies under which conditions (RELATION) and which signal property (PROPERTY) it is bound to. redstoneOutputs triggers a Redstone output when interacting directly with the signal via right-click. Optional. Multiple entries allowed.

"redstoneOutput": {"config(manual.TRUE)": "powered"}

- **remoteRedstoneOutputs** specifies under which conditions (RELATION) and which signal property (PROPERTY) it is bound to. remoteRedstoneOutputs triggers a Redstone output when interacting with the signal via a signal controller. Optional. Multiple entries allowed.

"remoteRedstoneOutput": {"config(manual.FALSE)": "powered"}

- **customRenderBoundingBox** specifies a custom render area in blocks if the signal is larger in X or Z direction. Optional. Default is just the Y height of the signal. Example provided with standard mod barriers.

"customRenderBoundingBox": [-1,0,-1,1,5,1]

- **seProperties** lists the individual signal properties (PROPERTY). Signal properties are the individual components of a signal that can be configured. Must be present. Can be an empty list [] or contain multiple entries.

- **name** is the internal name of the signal property (PROPERTY). Must be present. The name and its corresponding value (ENUM) are used in many other places.

{"name": "kombisignal_distant"}

- **enumClass** specifies the list of available settings for this property. Should be present if multiple options exist. For Boolean-only properties, enumClass is omitted.

"enumClass": "KSDistant"

- **defaultState** specifies the default setting of the property. Must be present. If enumClass is defined, a valid value from the option list must be provided. For Booleans, use „true“ or „false“.

"defaultState": "OFF"

or

"defaultState": true

- **changeableStage** specifies where this signal property can be modified. Should be present. Default is APISTAGE. Possible stages:
 - **APISTAGE**: Available in both the selection item and the signal controller.
 - **GUISTAGE**: Available only in the selection item.
 - **APISTAGE_NONE_CONFIG**: Available only in the signal controller.

"changeableStage": "GUISTAGE"

- **autoname** specifies whether the setting name in-game is assigned automatically or manually via the language file. Optional. Default is „false“.

"autoname": true

- **itemDamage** specifies how much damage the item takes when placing signals. Includes damage from the signal system and other signal properties. An item has 100 damage points. Optional. Default is 1. Only integers ≥ 0 allowed.

"itemDamage": 1

- **dependencies** specifies the conditions (RELATION) on which the signal property depends. Optional. See 1.4 for more details. Unused SEProperties are hidden in the signal controller.

"dependencies": "config(KOMBITYPE.DISTANT)"

5.2 Enum Definition / Configuration Options

Configuration options allow a signal property to have multiple values, for example, for signal aspects. To do this, create an **enums.json** file in the **enumdefinition/** folder, where lists of configuration options are defined.



The name of the list must be unique to avoid conflicts with other CPs.

```
1 {
2   "KSDistant": [
3     "OFF", "KS1", "KS1_BLINK", "KS2"
4   ],
5   "HP": [
6     "OFF", "HP0", "HP1", "HP2", "SHUNTING"
7   ]
8 }
```

Listing 2: Structure of the enums.json.

5.3 Creating a Placement Tool

You can create your own selection item to place your signal systems. To do this, create a **tools.json** file in the **tools/** folder, which contains a list of selection items. When registering a new selection item, you will need an item model and an item texture.

```
1 {
2   "placementtools": [ "PLACEMENT_TOOL" ]
3 }
```

Listing 3: Adding a Placement Tool in the tools.json.



This tool would now be stored internally as "placementtool". Usually, you only need one selection item. You can use an already existing one, but for clarity, we recommend creating a separate placement tool for your signal systems for each CP. Your tool should have a unique name to avoid conflicts with other CPs.

5.4 Models and Textures

At the current stage, our system only supports valid JSON block models. Therefore, we recommend using Blockbench for modeling. Textures can also be created and edited with Blockbench or with other image editing programs, such as GIMP. Since there are plenty of tutorials available online on how these programs work, we will not go into detail on how to create models and textures. We will, however, explain a few specific considerations related to JSON files.



It is best not to model the signal as a whole. All parts that will later be switchable, replaceable, or have other functions must be created as separate models. These are then combined into a signal with the respective configuration options in the model definition.

Textures should be saved in **textures/blocks/yourcontentpack/** and models in **models/block/yourcontentpack/** to avoid conflicts with other CPs. There are two special considerations for models: First, a tint index can be assigned to a texture, and second, textures can be replaced using the texture variable in the model definition (5.5) for swappable textures.

```
1 {
2   "textures": {
3     "1": "opensignals:blocks/default/shield_black",
4     "15": "opensignals:blocks/zs3/off",
5     "overlay": "opensignals:blocks/zs3/z10",
6     "particle": "opensignals:blocks/default/shield_black"
7   },
8   "elements": [
9     {
10      "name": "overlay",
11      "from": [5.5, 1, 3.9],
12      "to": [10.5, 8, 3.9],
13      "faces": {
14        "north": {"uv": [0, 0, 5, 7], "textures": "#overlay", "tintindex": 1}
15      }
16    },
17    {
18      "..."
19    }
20  ]
21 }
```

Listing 4: Special Features of the JSON Models.

5.4.1 Tint Index

The tint index is a „color index“ that allows textures to be recolored dynamically. The number refers to the respective entry in the **colors** list in the signal system (5.1), with the first entry having index 0.



This feature is generally used only for special cases and is not normally applied.

5.4.2 Texture Variables

Texture variables can be used to override textures for various settings. This is done in the model definition (5.5) by assigning a new texture to the specified texture variable. The texture variable can also be renamed for easier identification.

5.4.3 Emissive Textures

If OptiFine is installed, textures with a glow effect („emissive“) can be used. The corresponding textures must exist twice in the same folder, and one file must have „_e“ at the end of its name. Example: **texture.png** and **texture_e.png**. Additionally, the „Emissive-Texture“ option must be enabled in the OptiFine settings.



This feature is outdated and only recommended for Minecraft 1.12.2. The following method is NEW and only works in Minecraft 1.16+

Textures that should fully glow must include **lamp_** in the texture name. It is also possible to create weakly glowing textures, e.g., reflective textures, by including **reflection_** in the texture name.

5.5 Creating a Model Definition

In the model definition, the individual models are assembled into a complete signal system. Conditions can also be specified for when certain models should be displayed. Additionally, swappable textures can be defined.

```
1 {
2   "textures": {
3     "VAR...": "TEXTUREFILE"
4   },
5   "models": {
6     "MODELFILE...": {
7       "textures": [
8         {
9           "loadOFFstate": "BOOLEAN"
10          "blockstate": "RELATION",
11          "retexture": {
12            "VARIABLE...": "VAR || TEXTUREFILE"
13          },
14          "extentions": {
15            "EXTENTION.json...": {
16              "PROPERTY": "VARIABLE"
17            }
18          },
19          "x": "INT",
20          "y": "INT",
21          "z": "INT"
22        },
23        {
24          "... "
25        }
26      ],
27      "x": "INT",
28      "y": "INT",
29      "z": "INT"
30    }
31  }
32 }
```

Listing 5: Registering Models.

5.5.1 Explanations

- **textures** Here, texture files (TEXTUREFILE) can be stored in internal variables (VAR) so that the texture file does not have to be specified everywhere. The texture file is the path + filename. Optional. Can contain multiple entries.

"texture1": "opensignals:blocks/path/texture1"

- **models** Lists the individual models (MODELFILE) for the signal system. Conditions for block states and swappable textures can be defined for the models. Must be present. The MODELFILE consists of „folder“/„file“ relative to „models/blocks/“.

"models": {"ks/ks_ne2": {""... "}}

- **textures** A list of elements. Must be present. Can contain multiple elements or an empty list.

"textures": [{""... "}]

- **loadOFFstate** Specifies whether a given off state should be loaded. Optional. Default is „true“.

`"loadOFFstate": true`



It is important that if you set loadOFFstate to false, your EnumClass must include a value called OFF; otherwise, it will not work.

- **blockstate** Contains conditions (RELATION) for when this model or block state should be loaded. Must be present. Can be left empty. Methods are explained in more detail in 1.4.

`"blockstate": "with(KOMBISIGNAL.OFF)"`

`"blockstate": "has(KOMBITYPE)"`

`"blockstate": "hasandis(NE2)"`

- **retexture** Allows individual textures in a model to be replaced. This reduces, for example, multiple submodels for different signal aspects into a single base model. Optional. VARIABLE is the texture variable from the model. VAR is the internal texture variable defined under „textures“, or you can directly specify the texture file (TEXTUREFILE).

`"retexture": {"1": "texture1"}`

- **extentions** An extension system. Allows certain elements that are the same across multiple signal systems to be outsourced to separate files. The extension file (EXTENTION) is located in the **modeldefinitions/** folder and must be named as follows: **test.extention.json**. VARIABLE is the texture variable from the model. PROPERTY specifies the associated signal property. Optional. Can contain multiple entries.

`"extentions": {"test.json": {"TEST_PROPERTY": "3"}}`

- **x** Horizontal offset of the submodel or block state. Optional. Default is 0. Decimal values allowed.

`"x": 0`

- **y** Vertical offset of the submodel or block state. Should be present. Default is 0. Decimal values allowed.

`"y": 0`

- **z** Forward/backward offset of the submodel or block state. Optional. Default is 0. Decimal values allowed.

`"z": 0`

The following section introduces the extension file.

```

1 {
2   "extention": {
3     "ENUM...": "TEXTUREFILE"
4   }
5 }
```

Listing 6: Extensions file.

5.5.2 Extension File Explanations

- **extention** is an extension for the model definition. It allows certain elements from the model definition that are the same across multiple signal systems to be separated so they don't need to be listed multiple times. TEXTUREFILE is the path to the texture file. ENUM specifies the setting from the **enum/enums.json** list for which the texture applies. Can contain multiple entries. Not required.

```
"extention": {"WRW": "opensignals:blocks/mast_sign/wrw"}
```



We use this feature, for example, for mast signs, since they are present in every light signal system. Generally, this is not required and can be omitted for simplicity.

5.6 Adding Sounds

To add sounds to the signals, they must be placed in the **sounds/** folder.



It is recommended to also create a subfolder with the content pack name here to increase compatibility with other CPs.

Each sound must also be registered in the **sounds.json** file located in the **opensignals/** main folder.

```
1 {  
2   "SOUND": {"category": "block", "sounds": [{"name": "opensignals:CONTENTPACKNAME/FILE"}]},  
3   "andreas_cross": {"category": "block", "sounds": [{"name": "opensignals:andreas_cross"}]}  
4 }
```

Listing 7: Registering Sounds.

5.6.1 Explanations

- **SOUND** is the internal name of the sound.
- **CONTENTPACKNAME** It is recommended to create a subfolder with the content pack name to improve compatibility with other CPs.
- **FILE** is the sound file to be played.

5.7 Signal Configs for the Interlocking System

To make signals compatible with the interlocking system, the system must be informed which signal aspect should be displayed in which situation. First, the folder structure will be introduced, and then the individual systems will be explained. A separate file must be created for each signal system.

5.7.1 Folder Structure

The files for the interlocking systems are located in the following subfolders:

```
signalconfigs/  
├─ change/  
├─ default/  
├─ disable/  
├─ reset/  
├─ shunting/  
├─ subsidiary/  
└─ subsidiaryenums/
```

- **change/** contains information on what the signal should display according to the aspect of the next signal.
- **default/** contains information on what the signal should display when the next signal is a dummy signal.
- **disable/** contains information for when the current signal should switch to the marker light mode.
- **reset/** contains information on what the signal should display when the route is cleared.
- **shunting/** contains information on what the signal should display when a shunting route is set.
- **subsidiary/** contains information for signals with special assignments or additional signals.
- **subsidiaryenums/** allows adding new additional signals or signals with special assignments, provided they are not already defined elsewhere.

5.7.2 Reset Setting

The „reset“ setting is the configuration that is loaded when the signal's route is cleared. Therefore, a stop aspect should always be displayed here.

```
1 {  
2   "currentSignals": ["SIGNALSYSTEMS..."],  
3   "values": {  
4     "RELATION...": ["PROPERTY.ENUM..."],  
5     "..."  
6   }  
7 }
```

Listing 8: Reset Config.

- **currentSignals** is a list of signal systems for which this configuration applies.
- **values** is a mapping of interlocking settings to a list of signal properties (PROPERTY) and their corresponding value (ENUM). „true“ is always activated.



It is also recommended to set all other parts of the signal to an „OFF-State“. See the following example.

```

1 {
2   "currentSignals": ["hvsignal", "hvsignalbridge"],
3   "values": {
4     "true": ["stopsignal.HP0", "hphome.HP0", "hpblock.HP0", "distantSignal.VR0",
5             "zs3.OFF", "zs3v.OFF", "zs1.FALSE", "zs7.FALSE"],
6     "!config(HPTYPE.OFF)": ["distantSignal.OFF"]
7   }
8 }
```

Listing 9: Example of a Reset Configuration.

5.7.3 Default Setting

The „default“ setting is loaded when the next signal does not exist or is a dummy signal, meaning it does not provide any additional information.

```

1 {
2   "currentSignals": ["hvsignal", "hvsignalbridge"],
3   "values": {
4     "speed(<.1) || speed(>.15)": ["stopsignal.HP1", "hphome.HP1", "hpblock.HP1"],
5     "speed(==.1)": ["zs3.Z1", "stopsignal.HP2", "hphome.HP2", "hpblock.HP1"],
6     "speed(==.2)": ["zs3.Z2", "stopsignal.HP2", "hphome.HP2", "hpblock.HP1"],
7     "speed(==.3)": ["zs3.Z3", "stopsignal.HP2", "hphome.HP2", "hpblock.HP1"],
8     "speed(==.4)": ["stopsignal.HP2", "hphome.HP2", "hpblock.HP1"],
9     "speed(==.5)": ["zs3.Z5", "stopsignal.HP2", "hphome.HP2", "hpblock.HP1"],
10    "speed(==.6)": ["zs3.Z6", "stopsignal.HP2", "hphome.HP2", "hpblock.HP1"],
11    "speed(==.7)": ["zs3.Z7", "stopsignal.HP1", "hphome.HP1", "hpblock.HP1"],
12    "speed(==.8)": ["zs3.Z8", "stopsignal.HP1", "hphome.HP1", "hpblock.HP1"],
13    "speed(==.9)": ["zs3.Z9", "stopsignal.HP1", "hphome.HP1", "hpblock.HP1"],
14    "speed(==.10)": ["zs3.Z10", "stopsignal.HP1", "hphome.HP1", "hpblock.HP1"],
15    "speed(==.11)": ["zs3.Z11", "stopsignal.HP1", "hphome.HP1", "hpblock.HP1"],
16    "speed(==.12)": ["zs3.Z12", "stopsignal.HP1", "hphome.HP1", "hpblock.HP1"],
17    "speed(==.13)": ["zs3.Z13", "stopsignal.HP1", "hphome.HP1", "hpblock.HP1"],
18    "speed(==.14)": ["zs3.Z14", "stopsignal.HP1", "hphome.HP1", "hpblock.HP1"],
19    "speed(==.15)": ["zs3.Z15", "stopsignal.HP1", "hphome.HP1", "hpblock.HP1"],
20    "true": ["distantSignal.VR0", "zs3v.OFF"],
21
22    "zs6state(true)": ["zs3.ZS6"],
23    "zs2value(A)": ["zs3.A"],
24    "zs2value(B)": ["zs3.B"],
25    "zs2value(C)": ["zs3.C"],
26    "...": ["..."],
27    "zs2value(ZS6)": ["zs3.ZS6"],
28    "zs2value(ZS13)": ["zs3.ZS13"]
29   }
30 }
```

Listing 10: Example of a Default Configuration.

- **currentSignals** is a list of signal systems for which this configuration applies.
- **values** is a mapping of interlocking settings to a list of signal properties (PROPERTY) and their corresponding value (ENUM).



Here, the signal aspects should always be filtered according to the speed set in the route. $speed \leq 0$ and > 15 means no restriction. „true“ is always activated. Additionally, the values for subsidiary signals are set.

5.7.4 Disable Setting

The „disable“ setting is loaded when the current signal should display the marker light, for example, when it is being skipped.

```

1 {
2   "currentSignals": ["hvsignal", "hvsignalbridge"],
3   "values": {
4     "true": ["stopsignal.OFF", "hphome.OFF", "hpbblock.OFF", "distantSignal.VR0", "zs3.OFF",
5             "zs3v.OFF", "zs1.FALSE", "zs7.FALSE", "identifier.TRUE"],
6     "!config(HPTYPE.OFF)": ["distantSignal.OFF"]
7   }
8 }
```

Listing 11: Example of a Disable Configuration.

5.7.5 Change Setting

The „change“ setting is loaded when a target signal exists and provides information about its state, allowing the current signal to react accordingly. Depending on the information received, it must be configured how the current signal should respond.



The filename should ideally include the names of both the current and target signals for easy identification. Files can also be created to handle mixed signal systems, e.g., from HV (current) to KS (next) signals.

```

1 {
2   "currentSignals": ["SIGNALSYSTEMS..."],
3   "nextSignals": ["SIGNALSYSTEMS..."],
4   "savedPredicates": {
5     "VARIABLE...": "RELATION..."
6   },
7   "values": {
8     "map(VARIABLE)...": ["PROPPROPERTY.ENUM..."],
9
10    "signalRepeater(true)": ["PROPPROPERTY.ENUM..."],
11
12    "RELATION": ["PROPPROPERTY.ENUM..."],
13
14    "speed(XYZ)": ["PROPPROPERTY.ENUM..."],
15
16    "zs2value(XYZ)": ["PROPPROPERTY.ENUM..."],
17
18    "zs6state(true)": ["PROPPROPERTY.ENUM..."]
19  }
20 }
```

Listing 12: Change Config.

- **currentSignals** is a list of signal systems for which this configuration applies.
- **nextSignals** is a list of signal systems that the current signal announces ahead.
- **savedPredicates** allows conditions (RELATION) to be stored in custom-defined variables using the config() method. This is useful for code snippets that are repeated, so you don't have to rewrite them each time. Optional.
- **values** is a mapping of interlocking settings to a list of signal properties (PROPERTY) and their corresponding value (ENUM).
 - **map(VARIABLE)** calls the stored values (RELATION) from the respective variable. Uses information from the next signal.
 - **signalRepeater(true)** if the distant signal is a repeater, this can be set in the interlocking system. To display signal aspects correctly, the desired output must be specified. signalRepeater(true) can be combined with map(VARIABLE).
 - **RELATION** directly specifies the condition (RELATION) using the config() method. Uses information from the next signal.
 - **speed(XYZ)** applies to the speed set in the current route. Uses information from the current route.
 - **zs2value(XYZ)** sets the subsidiary signals for the current route. Uses information from the current route.
 - **zs6state(true)** indicates that the route leads to the opposite track and activates the Zs6 indicator.


```

1 {
2   "currentSignals": ["hvsignal", "hvsignalbridge"],
3   "nextSignals": ["hvsignal", "hvsignalbridge"],
4   "savedPredicates": {
5     "stop": "config(stopsignal.HP0) || config(stopsignal.SHUNTING)
6             || config(hphome.HP0) || config(hphome.HP0_ALTERNATE_RED) || config(hpblock.HP0)",
7     "drive": "config(stopsignal.HP1) || config(hphome.HP1) || config(hpblock.HP1)",
8     "slow": "config(stopsignal.HP2) || config(hphome.HP2)"
9   },
10  "values": {
11    "map(stop)": ["distantSignal.VR0", "zs3v.OFF"],
12    "map(drive)": ["distantSignal.VR1"],
13    "map(slow) && signalRepeater(true)": ["distantSignal.VR2"],
14
15    "(config(zs3.Z1) || config(zs3plate.Z1))": ["zs3v.Z1"],
16    "(config(zs3.Z2) || config(zs3plate.Z2))": ["zs3v.Z2"],
17    "...": ["..."],
18    "(config(zs3.Z15) || config(zs3plate.Z15))": ["zs3v.Z15"],
19
20    "speed(<.1) || speed(>.15)": ["stopsignal.HP1", "hphome.HP1", "hpblock.HP1"],
21    "speed(==.1)": ["zs3.Z1", "stopsignal.HP2", "hphome.HP2", "hpblock.HP1"],
22    "speed(==.2)": ["zs3.Z2", "stopsignal.HP2", "hphome.HP2", "hpblock.HP1"],
23    "...": ["..."],
24    "speed(==.15)": ["zs3.Z15", "stopsignal.HP1", "hphome.HP1", "hpblock.HP1"],
25
26    "zs6state(true)": ["zs3.ZS6"],
27    "zs2value(A)": ["zs3.A"],
28    "zs2value(B)": ["zs3.B"],
29    "zs2value(C)": ["zs3.C"],
30    "...": ["..."],
31    "zs2value(ZS6)": ["zs3.ZS6"],
32    "zs2value(ZS13)": ["zs3.ZS13"],
33
34    "config(zs3.A)": ["zs3v.A"],
35    "config(zs3.B)": ["zs3v.B"],
36    "config(zs3.C)": ["zs3v.C"],
37    "...": ["..."]
38  }
39 }

```

Listing 13: Example of a Change Configuration.

5.7.6 Shunting Setting

The „shunting“ setting is used for shunting signals and main signals with a shunting function to indicate shunting movements.

```

1 {
2   "currentSignals": ["hvsignal", "hvsignalbridge"],
3   "values": [
4     "stopsignal.SHUNTING"
5   ]
6 }

```

Listing 14: Example of a Shunting Configuration.

- **currentSignals** is a list of signal systems for which this configuration applies.
- **values** is a list of signal properties (PROPERTY) and their corresponding value (ENUM).

5.7.7 Subsidiary Setting

The „subsidiary“ setting specifies which additional signals should be displayed when activated. This applies to the subsidiary signals that can be directly activated via right-click on

the signal.

```
1 {
2   "currentSignal": "hvsignal",
3   "allStates": {
4     "ZS1": ["zs1.TRUE"],
5     "ZS7": ["zs7.TRUE"],
6     "ZS8": ["zs3.ZS8"],
7     "ZP9": ["zs3.ZP9"]
8   }
9 }
```

Listing 15: Example of a Subsidiary Configuration.

- **currentSignal** is the name of the current signal system.
- **allStates** specifies, for the various subsidiary signals, which signal properties (PROPERTY) and their corresponding values (ENUM) should be applied.

In „subsidiaryenums“ you can also add custom subsidiary signals in a separate file, which can be manually activated and defined in the Subsidiary Config for the respective signal systems. The default entries are always present. Multiple entries can be made.

```
1 {
2   "subsidiaryStates": [
3     {
4       "name": "NAME",
5       "showSubsidiaryAtSignal": "SIGNALASPECT",
6       "isCountable": "BOOLEAN"
7     },
8     {
9       "...": ""
10    }
11  ]
12 }
```

Listing 16: Structure of SubsidiaryEnums.

- **name** specifies the name of the subsidiary signal.
- **showSubsidiaryAtSignal** indicates how the signal should appear in the interlocking system when the subsidiary signal is active. The following values can be used: *SIGNAL_RED*, *SIGNAL_GREEN*, *SIGNAL_OFF*.
- **isCountable** specifies whether the counting mechanism should track the operation of this signal or not.

```
1 {  
2   "subsidiaryStates": [  
3     {  
4       "name": "Zs1",  
5       "showSubsidiaryAtSignal": "SIGNAL_RED",  
6       "isCountable": true  
7     }  
8   ]  
9 }
```

Listing 17: Example of SubsidiaryEnums.

5.8 Signal Bridges

To create custom signal bridges with your own blocks, they must be registered as described below.



If you want to create custom signals or signal systems for signal bridges, the entry

`"isBridgeSignal": true`

must be added to the SystemProperties in the signal system file.

The file can have any name and is structured as follows:

```
1 {
2   "BLOCKNAME": {
3     "type": "TYPE"
4   },
5   "...": {
6   }
7 }
```

Listing 18: Registering Signal Bridge Blocks.

- **blockname** specifies the name of the block. The name must be unique.
- **type** specifies the type of signal bridge block. The following values can be used: *BASE*, *MAST*, *MAST_HEAD*, *CANTILEVER*, *CANTILEVER_END*.

```
1 {
2   "signalbridge_mast": {
3     "type": "MAST"
4   },
5   "signalbridge_cantilever": {
6     "type": "CANTILEVER"
7   }
8 }
```

Listing 19: Example: Registering Signal Bridge Blocks.

Blockstate files must be created for the signal bridge blocks with the variants *facing = east*, *facing = north*, *facing = south*, *facing = west*. We will not go into further detail on blockstate files here. You can find more information online by looking at how a standard Minecraft resource pack is structured, or by examining existing blockstate files in the Signal Mod. The following example is provided:

```

1 {
2   "variants": {
3     "facing=east": {
4       "model": "opensignals:block/signalbridge/default/signalbridge_base",
5       "y": 90
6     },
7     "facing=north": {
8       "model": "opensignals:block/signalbridge/default/signalbridge_base"
9     },
10    "facing=south": {
11      "model": "opensignals:block/signalbridge/default/signalbridge_base",
12      "y": 180
13    },
14    "facing=west": {
15      "model": "opensignals:block/signalbridge/default/signalbridge_base",
16      "y": 270
17    }
18  }
19 }

```

Listing 20: Example of a Blockstate File for Signal Bridges

5.9 Creating Language Files

For translations and display names in the game, the internal language keys must be provided with translations in the language file. The English (**en_us**) file should always be included, as it is the Minecraft default. Additionally, you can provide other translations, such as German (**de_de**).



Note: For Minecraft version 1.12 and earlier, language files are saved in the **.lang** format. From Minecraft version 1.13 onwards, they use the **.json** format.



The best way to determine the language keys is to start the game without these files and carefully observe your signals both in the selection item and in the signal controller, noting down the “keys” as they appear.

```
1 {  
2   "block.opensignals.kssignal": "Ks - Combination signal"  
3 }
```

Listing 21: Language File in JSON Format.

```
1 tile.kssignal.name=Ks Signal
```

Listing 22: Language File in .lang Format.

5.10 Creating Animations

Signals can also be animated, meaning that when a mechanical signal performs a movement, it can be represented using this system.

There are the following types of animations:

- **ROTATION**, for rotations/turns
- **TRANSLATION**, for translations/movements

Basically, for each animated signal system, a JSON file with the same name is created in the *animations/* directory.

This file then contains, as already known from the model definitions, the individual models of the signal that should perform the movement.

```
1 {
2   "animations": {
3     "MODELFILE": {
4       "translationX": "FLOAT",
5       "translationY": "FLOAT",
6       "translationZ": "FLOAT",
7       "pivotX": "FLOAT",
8       "pivotY": "FLOAT",
9       "pivotZ": "FLOAT",
10      "animationConfigs": [
11        {
12          "predicate": "RELATION",
13          "mode": "ROTATION",
14          "rotationAxis": "AXIS",
15          "rotation": "DEGREE",
16          "animationSpeed": "FLOAT"
17        },
18        {
19          "predicate": "RELATION",
20          "mode": "TRANSLATION",
21          "destY": "FLOAT",
22          "animationSpeed": "FLOAT"
23        }
24      ]
25    },
26    "...":
27  }
28 }
```

Listing 23: Structure of the Animation Data

5.10.1 Explanations

- **animations** is simply the keyword for animations. Must be present.
- **MODELFILE** Lists each individual model (MODELFILE) that should be animated in the signal system. Conditions for block states and the animation itself can also be defined. Must be present. The MODELFILE consists of „folder“/„file“relative to „models/blocks/“.
- **translationX, translationY, translationZ** defines the displacement of the model within the signal along the three axes. Must be present for Translation and Rotation. Decimal numbers are allowed. Units are in blocks/meters.
- **pivotX, pivotY, pivotZ** defines the displacement of the model's pivot point along the three axes. Must be present and only applicable for ROTATION. Decimal numbers are allowed. Units are in blocks/meters. The pivot is always centered within the block.
- **animationConfigs** is simply the keyword for the type and additional properties of the animation. Must be present.

- **predicate** is analogous to the „blockstate“ in the model definitions (5.5) and contains conditions (RELATION) for when this model or block state should be loaded. Must be present. Can be left empty. Methods are explained in detail in 1.4.

"blockstate": `"with(KOMBISIGNAL.OFF)"`

"blockstate": `"has(KOMBITYPE)"`

"blockstate": `"hasandis(NE2)"`

- **mode** specifies the type of animation, either ROTATION or TRANSLATION. Must be present.

1. ROTATION

- **rotationAxis** specifies the axis around which the model should rotate. Must be present. Possible values are "X", "Y", "Z".
- **rotation** specifies the angle in degrees by which the model should rotate. Must be present.

2. TRANSLATION

- **destX, destY, destZ** specifies how far the model should move along the given axis. Must be present. Decimal numbers are allowed. Units are in blocks/meters.

- **animationSpeed** specifies the speed at which the animation should play. Optional. Decimal numbers are allowed.


```

1 {
2   "animations": {
3     "semaphore_signals/sema_main_wing1": {
4       "translationX": 0.6875,
5       "translationY": 7.5625,
6       "translationZ": 0,
7       "pivotX": -0.6875,
8       "pivotY": -0.5625,
9       "pivotZ": 0,
10      "animationConfigs": [
11        {
12          "predicate": "with(WING1.FALSE) && with(SEMATYPE.MAIN)",
13          "mode": "ROTATION",
14          "rotationAxis": "Z",
15          "rotation": 0
16        },
17        {
18          "predicate": "with(WING1.TRUE) && with(SEMATYPE.MAIN)",
19          "mode": "ROTATION",
20          "rotationAxis": "Z",
21          "rotation": -45
22        }
23      ]
24    },
25    "semaphore_signals/sema_distant_lamp1": {
26      "translationX": 0,
27      "translationY": 2,
28      "translationZ": 0,
29      "animationConfigs": [
30        {
31          "predicate": "with(SEMATYPE.DIST) && (with(SEMAVR.VR0)
32          || with(SEMAVR.VR2))",
33          "mode": "TRANSLATION",
34          "animationSpeed": 0.25,
35          "destY": 0
36        },
37        {
38          "predicate": "with(SEMATYPE.DIST) && with(SEMAVR.VR1)",
39          "mode": "TRANSLATION",
40          "animationSpeed": 0.25,
41          "destY": -0.25
42        }
43      ]
44    }
45  }
46 }

```

Listing 24: Example of Animation Data

6 Adding the Content Pack to the Game

The CP must be provided as a compressed zip file in **minecraft/contentpacks/opensignals/** in order to be loaded into the game. If you are playing on a server, the CP must be present on both the client and the server.

7 Bugs and Errors

If bugs or errors occur, it is helpful to first check the log file yourself to determine where the problem comes from. Most often, missing brackets or „ “ in a JSON file are the cause. If you cannot solve the issue or have questions, you can contact us on our Discord support forum. Please describe your error in detail and attach the log file, the error message, and the faulty CP.

8 Publishing a Content Pack

We will provide an official recommendation for publishing at a later time. What we can already recommend is to upload the CP as a resource pack on CurseForge. It is desirable that you share your CP with the community, as our mod is also open source and shared publicly. If you notify us, we will also link your pack on our website , provided it meets our conventions.